

COMP2522

Lesson 9

Streaming

Next Quiz

- Note: you can bring a cheatsheet: 1 sided, hand written

The Stream as a Factory Assembly Line

- Imagine you have a factory conveyor belt moving objects (your data) through a series of stations (the stream operations).
- Each station performs a specific task:

• Java Stream Step	Factory Analogy	Example
• -----	-----	-----
• stream()	Turning on the conveyor belt	Start moving items down the line
• filter()	A quality-check station	Only let through items that pass inspection (e.g., only even numbers)
• map()	A transformation station	Convert into something new (e.g., double each number; turn objects into their names)
• sorted()	A sorting station	Arrange items neatly on the belt before final packaging
• distinct()	A duplicate remover	Toss out duplicate products
• limit() / skip()	A cutoff gate	Only take the first N items or skip the first few
• collect()	The packing station	Gather all finished products into a box (e.g., a List, Set, or Map)

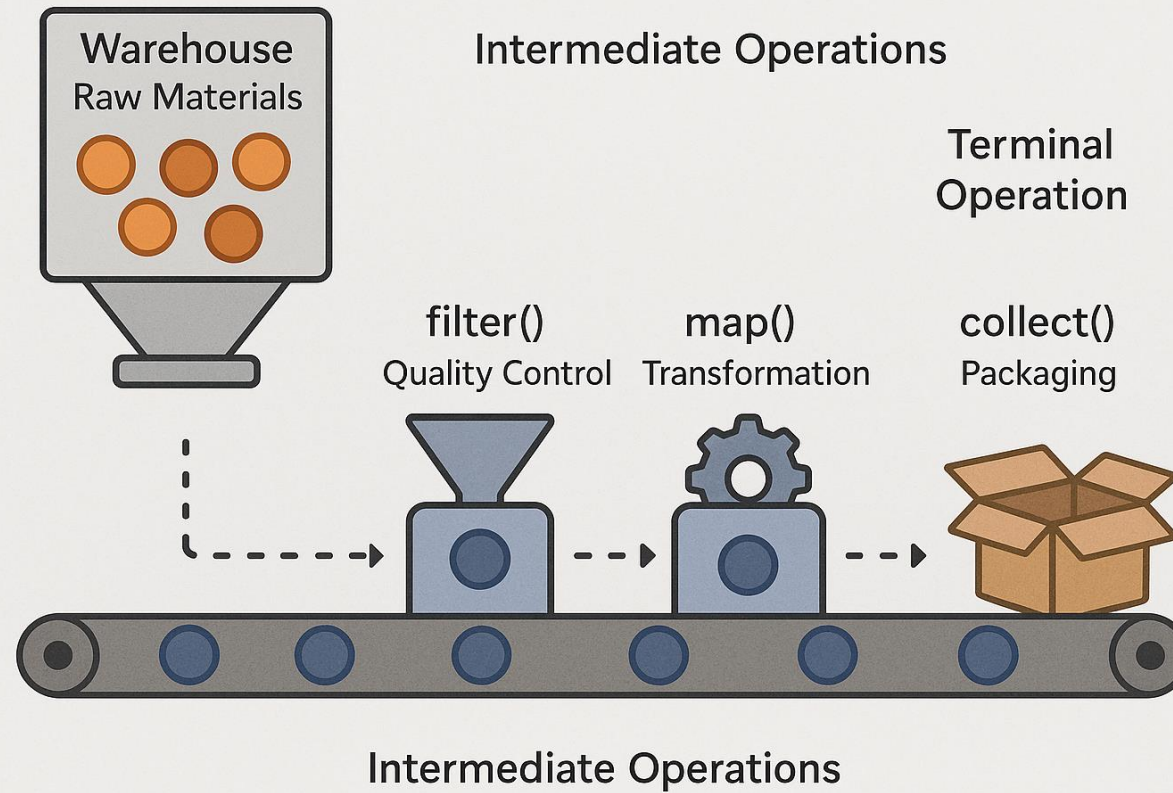
Example

```
final List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);
```

```
final List<Integer> result = numbers .stream()  
                                     .filter(n -> n % 2 == 0)      // Only even numbers pass the filter  
                                     .map(n -> n * n)              // Each even number gets squared  
                                     .collect(Collectors.toList()); // Box up the final products
```

- In words:
- You start a conveyor belt with raw materials (1–6).
- A filter worker removes odd numbers.
- A transformer worker squares each remaining number.
- Finally, a packer collects the finished goods into a box.

Analogy



Intermediate and Terminal Operations

- Extend the factory conveyor belt analogy to include intermediate and terminal operations.
- 1. Raw Materials
 - Your collection (e.g., List, Set) is a warehouse full of raw materials waiting to be processed.
 - When you call `.stream()`, you start the conveyor belt — items begin to move down the line, one by one.
- 2. Intermediate Operations → The Workstations
 - Each intermediate operation is a workstation on the assembly line.
 - They transform or filter the items but don't produce the final box yet — the belt keeps moving.
 - These operations are "**lazy**": they set up the stations but don't actually run until the belt is told to deliver the final goods.
- | Operation | Analogy | Description |
|-------------------------|--------------------------------|--|
| ----- | ----- | ----- |
| <code>filter()</code> | Quality control station | Only lets certain items pass (e.g., even numbers only) |
| <code>map()</code> | Transformation station | Changes each item (e.g., doubles the number) |
| <code>sorted()</code> | Sorting station | Reorders items before packaging |
| <code>distinct()</code> | Duplicate removal station | Tosses out any duplicates |
| <code>peek()</code> | Inspector's observation window | Lets you look at items passing by, but doesn't change them |
- These stations are set up in sequence, but nothing actually happens yet — the machines are idle until someone at the end says, "Start the line!"
- 3. Terminal Operation → The Packaging Department
 - A terminal operation is the final step — it says,
 - "Now run the belt, process everything, and hand me the results!"
 - It's what triggers the whole assembly line to start working.

More: Intermediate and Terminal Operations

- | • Operation | Analogy | What It Does |
|-----------------------------|--------------------------|---|
| • ----- | ----- | ----- |
| • collect() | Packing items into boxes | Gathers the finished products into a List, Set, or Map |
| • forEach() | Shipping clerk | Takes each product and sends it off (prints, saves, etc.) |
| • count() | Inventory counter | Counts how many finished products were made |
| • findFirst() / findAny() | First item inspector | Grabs the first acceptable product |
| • reduce() | Combiner | Merges all products into one (e.g., sum of all items) |
- Once the terminal operation runs, the belt stops — the factory can't reuse the same stream. You'd have to start a new one.
 - Summary:
 - > We roll raw materials (the integers 1–6) onto the belt.
 - > Odd ones fail inspection and are tossed out.
 - > Even ones go through a machine that squares them.
 - > The sorted results are boxed up neatly at the end.
 - > Once boxed, the belt stops.

Collection Streaming

- Streaming makes use of the Collection interface's **stream()** method and the Stream interface, and also lambda expressions
- [https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/Collection.html#stream\(\)](https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/Collection.html#stream())
- <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/stream/Stream.html>
- Can greatly streamline our code...replacing sometimes long, complex loops and conditionals
- Example streaming: filter, map, reduce, forEach
- Filter allows us to use predicates (an interface, similar to a function) to filter IN data
- E.g. keep all Strings in the list that start with "w"
- E.g. keep all even numbers in the collection
- Map allows us to convert between types
- E.g. map an integer to a String; map a Person to a String
- Reduce allows us to reduce a larger amount of data to a smaller amount
- E.g. reduce a list of numbers to their sum
- forEach allows us to process a collection of data, via a lambda

stream()

- Streaming and filtering can be used to replace for loops and if statements, with data from array or collections.
 - Process, transform and compute data from collections using functional programming techniques
 - The stream() method works on arrays and collections. it's a SEQUENCE of operations on the data, one after another
 - Process data collections and create subcollections or do "data mining"
 - Streams do not affect their original source (those are immutable)
-
- without streaming:

```
List<String> words = Arrays.asList("abc", "def", "ghi", "axe", "after", "whatever");
List<String> words_with_a = new ArrayList();
for(final String word: words){
    if(word != null && word.startsWith("a")){
        words_with_a.add(word.toUpperCase());
    }
}
```
 - with streaming:

```
List<String> words = Arrays.asList("abc", "def", "ghi", "axe", "after", "whatever");
List<String> words_with_a = words.stream()
    .filter(word->word != null)
    .filter(word->word.startsWith("a"))
    .map(String::toUpperCase)
    .toList();
```
-
- 1. there is a data source (a collection or array), whose .stream() method we execute
 - 2. there are intermediate operations such as filtering, mapping, and sorting. these operations are "lazy" which means they don't execute until a terminal operation is called (remember this; it affects the code)
 - 3. there are terminal operations that produce a result (e.g. sum(), count(), average(), min(), max(), toList()) or a side effect (e.g. printed).
 - The result of streaming and filtering is code that clean, efficient, short and readable. it is measured as lower complexity than having loops and decisions.

Foreach with Lambda Expression

```
List<String> notes;  
notes = new ArrayList<>();  
notes.add("buy groceries");  
notes.add("get enough sleep");  
notes.add("have fun");  
notes.add("work harder");  
  
notes.forEach(n -> System.out.println("note is " + n.toUpperCase()));
```

Foreach improved

```
List<String> notes;  
Consumer<String> consumer; // this variable will “contain” a function
```

```
notes = new ArrayList<>();  
notes.add("buy groceries");  
notes.add("get enough sleep");  
notes.add("have fun");  
notes.add("work harder");
```

```
consumer = n -> System.out.println(n.toUpperCase() + "!!");
```

```
notes.forEach(consumer); // apply the above function to this collection  
// or notes.stream().forEach(consumer);
```

stream(), filter(), etc....Intermediate Operations

- Intermediate Operations (Zero or more operations):
 - These operations do not consume the stream and can be chained. They return another stream, allowing for more transformations.
-
- | | |
|--|---|
| • 1. filter(Predicate<T> predicate): | Filters elements based on a condition. |
| • 2. map(Function<T, R> mapper): | Transforms elements. |
| • 3. flatMap(Function<T, Stream<R>> mapper): | Transforms and flattens streams. |
| • 4. distinct(): | Removes duplicate elements. |
| • 5. sorted(): | Sorts elements in natural order. |
| • 6. sorted(Comparator<? super T> comparator): | Sorts using a custom comparator. |
| • 7. limit(long maxSize): | Limits the stream to a given number of elements. |
| • 8. skip(long n): | Skips the first n elements. |
| • 9. peek(Consumer<? super T> action): | Allows peeking into elements. |
| • 10. takeWhile(Predicate<T> predicate): | Takes elements while the condition is true. |
| • 11. dropWhile(Predicate<T> predicate): | Drops elements while the condition is true. |
| • 12. parallel(): | Converts the stream into a parallel stream. |
| • 13. sequential(): | Converts a parallel stream back into sequential. |
| • 14. unordered(): | Makes the stream unordered (useful for parallel performance). |
| • 15. flatMapToInt(Function<? super T, ? extends IntStream> mapper): | Maps to IntStream. |
| • 16. flatMapToLong(Function<? super T, ? extends LongStream> mapper): | Maps to LongStream. |
| • 17. flatMapToDouble(Function<? super T, ? extends DoubleStream> mapper): | Maps to DoubleStream. |
| • 18. mapToInt(ToIntFunction<? super T> mapper): | Maps to IntStream. |
| • 19. mapToLong(ToLongFunction<? super T> mapper): | Maps to LongStream. |
| • 20. mapToDouble(ToDoubleFunction<? super T> mapper): | Maps to DoubleStream. |

stream(), filter(), etc...Terminal Operations

- Terminal Operations (One per stream):
- These consume the stream and produce a result or a side-effect.
- 1. `forEach(Consumer<? super T> action):` Performs an action on each element.
- 2. `forEachOrdered(Consumer<? super T> action):` Like `forEach()`, but maintains order in parallel streams.
- 3. `toArray():` Converts the stream to an array.
- 4. `reduce(T identity, BinaryOperator<T> accumulator):` Reduces elements to a single value using an identity value and accumulator.
- 5. `reduce(BinaryOperator<T> accumulator):` Reduces elements using an accumulator.
- 6. `collect(Collector<? super T, A, R> collector):` Collects the elements into a collection or structure.
- 7. `collect(Supplier<R> supplier, BiConsumer<R, ? super T> accumulator, BiConsumer<R, R> combiner):` Alternative custom collection.
- 8. `min(Comparator<? super T> comparator):` Returns the minimum element according to a comparator.
- 9. `max(Comparator<? super T> comparator):` Returns the maximum element according to a comparator.
- 10. `count():` Counts the elements in the stream.
- 11. `anyMatch(Predicate<? super T> predicate):` Returns true if any element matches the condition.
- 12. `allMatch(Predicate<? super T> predicate):` Returns true if all elements match the condition.
- 13. `noneMatch(Predicate<? super T> predicate):` Returns true if no elements match the condition.
- 14. `findFirst():` Returns the first element, if present.
- 15. `findAny():` Returns any element, typically used with parallel streams.
- 16. `iterator():` Returns an iterator for the stream.
- 17. `spliterator():` Returns a spliterator for the stream.

Streams as Data

- Instead of repeating patterns of streams, create methods to contain that code
- For example if we had a Collection of Province references, we could create a sub-list of them, filtering out null provinces and null province names using the "filteredStream" method below
- This example prints all the provinces whose names aren't blank, in descending ("reversed") alphabetical order

```
final List<Province> all = filteredStream(provinces)
    .filter(p->!p.getName().isBlank())
    .sorted(Comparator.comparing(Province::getName).reversed())
    .toList();
all.forEach(System.out::println);

// reusable method to get non-null province stream
private static Stream<Province> filteredStream(final List<Province> provinces)
{
    return provinces.stream().filter(p-> p != null && p.getName() != null);
}
```

Let's look at all of the major methods, starting with the terminal operations.

forEach(Consumer<? super T> action)

- Performs an action on each element in the stream; commonly used for printing or modifying each element.
- // Printing each element in a list.
- `List<String> names = List.of("Alice", "Bob", "Charlie");`
- `names.stream().forEach(System.out::println);`
- // Adding each element to another list.
- `List<String> copy = new ArrayList<>();`
- `names.stream().forEach(copy::add);`
- Note: `forEachOrdered` is similar, but in a parallel stream, it ensures the elements are processed in the encounter order (the order they appear in the stream source).

toArray()

- Converts the elements in the stream to an array, which is useful when you need a fixed-size, indexed collection of elements.
- // Converting a list of strings to an array.
- `List<String> names = List.of("Alice", "Bob", "Charlie");`
- `String[] array = names.stream().toArray(String[]::new);`
- // Converting a stream of integers to an integer array.
- `int[] array = IntStream.range(1, 4).toArray(); // [1, 2, 3]`

reduce(T identity, BinaryOperator<T> accumulator)

- This operation combines all elements into a single result, starting with an initial value ("identity") and using an accumulator function to combine them
- // Summing integers with an initial value: 16
- `int sum = Stream.of(1, 2, 3).reduce(10, Integer::sum);`
- // Concatenating strings with an initial prefix: Names: AliceBob
- `String result = Stream.of("Alice", "Bob").reduce("Names: ", String::concat);`
- or
- `String result = Stream.of("Alice", "Bob").reduce("Names:\n ", (acc, name)->acc + name + System.lineSeparator()); // each name on its own new line`

reduce(BinaryOperator<T> accumulator)

- Similar to the previous example, but with no initial value
- `// Summing integers: sum.get() is 6; sum is Optional[6]`
- `Optional<Integer> sum = Stream.of(1, 2, 3).reduce(Integer::sum);`
- or
- `List<Integer> numbers = List.of(1, 2, 3);`
- `Optional<Integer> sum = numbers.stream().reduce(Integer::sum);`

- `// Finding the longest string: Charlie`
- `Optional<String> longest = Stream.of("Alice", "Bob", "Charlie").reduce((a, b) -> a.length() > b.length() ? a : b);`
- or
- `List<String> names = List.of("Alice", "Bob", "Charlie");`
- `Optional<String> longest = names.stream().reduce((a, b) -> a.length() > b.length() ? a : b);`

Optionals

- **Optional** is a container that might or might not hold a value
 - Used to avoid **null** and **NullPointerException**
 - When using methods like **reduce()** with no initial value, the result may be absent if the stream is empty. Optional lets us safely check for a value.
-
- **List<Integer> numbers = List.of(); // Empty list, or possibly-empty list**
 - **Optional<Integer> sum = numbers.stream().reduce(Integer::sum);**
-
- **if (sum.isPresent()) {**
 - **System.out.println("Sum: " + sum.get());**
 - **} else {**
 - **System.out.println("No result, stream was empty.");**
 - **}**
-
- **isPresent()** checks if there's a value.
 - **get()** retrieves the value if present.
-
- In this example, sum is empty because the list has no elements, so **isPresent()** is false and we avoided null issues.
 - Use **Optional** as a return type for methods that might return a value but could also be empty, to explicitly indicate "no result" instead of returning null; however, do not use Optional for fields in classes or as parameters to methods, as it is designed specifically for return values to signal the potential absence of a result.

collect(Collector<? super T, A, R> collector)

- Collects the elements of the stream into a collection or other data structure like a List, Set, or Map
- // Collecting to a list
- `List<String> names = Stream.of("Alice", "Bob").collect(Collectors.toList());`
- // Collecting to a set.
- `Set<String> uniqueNames = Stream.of("Alice", "Alice", "Bob").collect(Collectors.toSet());`

collect(Supplier<R> supplier, BiConsumer<R, ? super T> accumulator, BiConsumer<R, R> combiner)

- A more customizable way of collecting elements by specifying how to create, accumulate, and combine the results
- `// Collecting to a custom list`
- `List<String> names = Stream.of("Alice", "Bob").collect(ArrayList::new, List::add, List::addAll);`
- `// Collecting to a map with custom logic`
- `Map<String, Integer> nameLengths = Stream.of("Alice", "Bob").collect(HashMap::new, (map, name) -> map.put(name, name.length()), Map::putAll);`
- `for(final Map.Entry<String, Integer> entry : nameLengths.entrySet()){`
- `System.out.println("Name: " + entry.getKey() + ", Length: " + entry.getValue());`
- `}`
- `// Name: Bob, Length: 3`
- `// Name: Alice, Length: 5`

min(Comparator<? super T> comparator)

- Finds the minimum element in the stream based on a specified comparator.
- // Finding the minimum integer: 1
- `Optional<Integer> min = Stream.of(3, 1, 4).min(Integer::compare);`
- // Finding the shortest string: Bob
- `Optional<String> shortest = Stream.of("Alice", "Bob", "Charlie")`
- `.min(Comparator.comparingInt(String::length));`

`max(Comparator<? super T> comparator)`

- Finds the maximum element in the stream based on a specified comparator.
- `// Finding the maximum integer: 4`
- `Optional<Integer> max = Stream.of(3, 1, 4).max(Integer::compare);`
- `// Finding the longest string: Charlie`
- `Optional<String> longest = Stream.of("Alice", "Bob", "Charlie")`
- `.max(Comparator.comparingInt(String::length));`

count()

- Counts the number of elements in the stream
- `// Counting elements in a stream: 3`
- `long count = Stream.of(1, 26, 3).count();`
- `// Counting words with more than 3 letters: 2`
- `long count = Stream.of("Alice", "Bob", "Charlie").filter(name -> name.length() > 3).count();`

anyMatch(Predicate<? super T> predicate)

- Checks if any element in the stream matches the given condition
- // Checking if any number is even: true
- `boolean hasEven = Stream.of(1, 2, 3).anyMatch(n -> n % 2 == 0);`
- // Checking if any name starts with "A": true
- `boolean hasA = Stream.of("Alice", "Bob").anyMatch(name -> name.startsWith("A"));`

allMatch(Predicate<? super T> predicate)

- Checks if all elements in the stream match the given condition
- `// Checking if all numbers are even: true`
- `boolean allEven = Stream.of(2, 4, 6).allMatch(n -> n % 2 == 0);`
-
- `// Checking if all names are non-empty: true`
- `boolean allNonEmpty = Stream.of("Alice", "Bob").allMatch(name -> !name.isEmpty());`

`noneMatch(Predicate<? super T> predicate)`

- Checks if no elements in the stream match the given condition
- `// Checking if no numbers are negative: true`
- `boolean noNegative = Stream.of(1, 2, 3).noneMatch(n -> n < 0);`
- `// Checking if no name starts with "Z": true`
- `boolean noZ = Stream.of("Alice", "Bob").noneMatch(name -> name.startsWith("Z"));`

findFirst()

- Returns the first element in the stream, if it exists
- Useful in ordered streams
- `// Finding the first element in a list: 6`
- `Optional<Integer> first = Stream.of(6, 1, 2, 3).findFirst();`
- `// Finding the first name that starts with "B": Bob`
- `Optional<String> firstB = Stream.of("Alice", "Bob", "Charlie")`
- `.filter(name -> name.startsWith("B"))`
- `.findFirst();`

iterator()

- Returns an iterator for the stream, allowing elements to be processed one by one manually
- // Getting an iterator from a stream
- `Iterator<String> iterator = Stream.of("Alice", "Bob").iterator();`
- `while (iterator.hasNext()) {`
- `System.out.println(iterator.next());`
- `}`
- // Using iterator to manually process elements
- `Iterator<Integer> iterator = Stream.of(1, 2, 3).iterator();`
- `iterator.forEachRemaining(System.out::println);`

Iterate through a text file

- `List<String> countries = Files.readAllLines(Paths.get("countries.txt"));`
- `countries.stream().iterator().forEachRemaining(System.out::println);`

Intermediate Operations

- Now that we know all the possible ways a stream can terminate, let's look at how the streams can first be processed
- In Java's Stream API, intermediate operations (like those we are about to study) are lazy, meaning they don't process data immediately; instead, they set up a pipeline that is activated only when a terminal operation is called
- Terminal operations trigger the entire pipeline, producing a final result or side effect and consuming the stream

filter(Predicate<T> predicate)

- Filters elements in the stream based on a given condition, **keeping** only those that match the condition
- // Filtering even numbers: 2, 4
- `Stream.of(1, 2, 3, 4).filter(n -> n % 2 == 0).forEach(System.out::println);`
- // Filtering (i.e. keeping) strings with more than 3 letters: apple, banana
- `Stream.of("apple", "pie", "banana")`
- `.filter(s -> s.length() > 3)`
- `.forEach(System.out::println);`

map(Function<T, R> mapper)

- Transforms each element in the stream by applying a function, changing it into another form
- `// Converting numbers to their squares: 1, 4, 9`
- `Stream.of(1, 2, 3).map(n -> n * n).forEach(System.out::println); // Output: 1, 4, 9`
- `// Converting strings to uppercase: APPLE, BANANA`
- `Stream.of("apple", "banana").map(String::toUpperCase).forEach(System.out::println);`
- `// Map each string to its first letter in uppercase: A, B, P, B`
- `Stream.of("apple", "banana", "peach", "blueberry")`
- `.map(word -> word.substring(0, 1).toUpperCase())`
- `.forEach(System.out::println);`

flatMap(Function<T, Stream<R>> mapper)

- Maps each element to its own stream, then flattens the result into a single stream (i.e. combines all the streams)
- // Splitting phrases into words: [he, , o wor, d, see you, ater]
- `Stream.of("hello world", "see you later").flatMap(word -> Stream.of(word.split("l"))).forEach(System.out::println);`
- // Flattening a list of lists: 1 2 3 4
- `List<List<Integer>> listOfLists = List.of(List.of(1, 2), List.of(3, 4));`
- `listOfLists.stream().flatMap(List::stream).forEach(System.out::println);`

distinct()

- Removes duplicate elements from the stream
- // Removing duplicate numbers: 1,2,3
- `Stream.of(1, 2, 2, 3, 3).distinct().forEach(System.out::println);`
- // Removing duplicate words: apple, banana
- `Stream.of("apple", "banana", "apple").distinct().forEach(System.out::println);`

sorted()

- Sorts elements in their natural order (i.e. ascending numerical order, alphabetical order, or as stated by the Comparable's compareTo() implementation)
- `// Sorting numbers in ascending order: 1, 2, 3, 4`
- `Stream.of(3, 1, 4, 2).sorted().forEach(System.out::println);`
- `// Sorting strings alphabetically: apple, banana, cherry`
- `Stream.of("banana", "apple", "cherry").sorted().forEach(System.out::println);`

sorted(Comparator<? super T> comparator)

- Sorts elements using a custom comparator
- // Sorting numbers in descending order: 4, 3, 2, 1
- `Stream.of(3, 1, 4, 2).sorted(Comparator.reverseOrder()).forEach(System.out::println);`
- // Sorting strings by length: pie, apple, banana
- `Stream.of("apple", "banana", "pie")
 .sorted(Comparator.comparingInt(String::length)).forEach(System.out::println);`
- // Sorting strings by length, descending order: banana, apple, pie
- `Stream.of("apple", "banana", "pie")
 .sorted(Comparator.comparingInt(String::length).reversed())
 .forEach(System.out::println);`

limit(long maxSize)

- Limits the stream to a given number of elements
- // Limiting to the first 2 numbers: 1, 2
- `Stream.of(1, 2, 3, 4).limit(2).forEach(System.out::println);`
- // Limiting to the first 3 strings: apple, banana, cherry
- `Stream.of("apple", "banana", "cherry", "date").limit(3).forEach(System.out::println);`
- // Limit to "half the size of the stream"; **note that we cannot directly count the stream elements without consuming them!**
- `List<Integer> numbers = List.of(1, 2, 3, 4);`
- `int halfSize = numbers.size() / 2;`
- `numbers.stream().limit(halfSize).forEach(System.out::println);`

skip(long n)

- Skips the first n elements in the stream
- // Skipping the first 2 numbers: 3, 4
- `Stream.of(1, 2, 3, 4).skip(2).forEach(System.out::println);`
- // Skipping the first string: banana, cherry
- `Stream.of("apple", "banana", "cherry").skip(1).forEach(System.out::println);`

peek(Consumer<? super T> action)

- View each element in the stream without modifying it (used for debugging)
- `// Printing each element during processing`
- `Stream.of(1, 2, 3)`
- `.peek(System.out::println)`
- `.map(n -> n * 2)`
- `.forEach(System.out::println);`
- `// Logging elements before filtering and saving in new List`
- `List<String> list = Stream.of("apple", "banana", "cherry")`
- `.peek(System.out::println)`
- `.filter(s -> s.length() > 5)`
- `.toList();`

takeWhile(Predicate<T> predicate)

- Takes elements from the stream as long as they match a condition, stopping when it fails
- // Taking numbers while they are less than 3: 1, 2
- Stream.of(1, 2, 3, 4)
- .takeWhile(n -> n < 3)
- .forEach(System.out::println);
- // Taking words while they start with "a": apple, apricot
- Stream.of("apple", "apricot", "banana")
- .takeWhile(s -> s.startsWith("a"))
- .forEach(System.out::println);

dropWhile(Predicate<T> predicate)

- Drops elements from the stream while they match a condition, resuming once it fails
- // Dropping numbers while they are less than 3: 3, 4
- `Stream.of(1, 2, 3, 4).dropWhile(n -> n < 3)`
- `.forEach(System.out::println);`
- // Dropping words while they start with "a": banana
- `Stream.of("apple", "apricot", "banana")`
- `.dropWhile(s -> s.startsWith("a"))`
- `.forEach(System.out::println);`

`flatMapToInt(Function<? super T, ? extends IntStream> mapper)`

- Maps each element to an `IntStream`, then flattens it into a single `IntStream`
- `// Flattening ranges of numbers: 0, 0, 1`
- `Stream.of(1, 2)`
- `.flatMapToInt(n -> IntStream.range(0, n))`
- `.forEach(System.out::println);`
- `// Converting lists of integers to a single IntStream: 1, 2, 3, 4`
- `Stream.of(List.of(1, 2), List.of(3, 4))`
- `.flatMapToInt(List::stream)`
- `.forEach(System.out::println);`

`flatMapToLong(Function<? super T, ? extends LongStream> mapper)`

- Maps each element to a LongStream, then flattens it into a single LongStream
- `// Flattening ranges of long numbers: 0, 0, 1`
- `Stream.of(1L, 2L)`
- `.flatMapToLong(n -> LongStream.range(0, n))`
- `.forEach(System.out::println);`
- `// Converting lists of longs to a single LongStream: 1, 2, 3, 4`
- `Stream.of(List.of(1L, 2L), List.of(3L, 4L))`
- `.flatMapToLong(List::stream)`
- `.forEach(System.out::println);`

`flatMapToDouble(Function<? super T, ? extends DoubleStream> mapper)`

- Maps each element to a `DoubleStream`, then flattens it into a single `DoubleStream`
- `// Flattening ranges of double numbers: 1.0, 1.5, 2.0, 2.5`
- `Stream.of(1.0, 2.0)`
- `.flatMapToDouble(n -> DoubleStream.of(n, n + 0.5))`
- `.forEach(System.out::println);`
- `// Converting lists of doubles to a single DoubleStream: 1.0, 2.0, 3.0, 4.0`
- `Stream.of(List.of(1.0, 2.0), List.of(3.0, 4.0))`
- `.flatMapToDouble(List::stream)`
- `.forEach(System.out::println);`

mapToInt(ToIntFunction<? super T> mapper)

- Maps each element to an int, resulting in an IntStream
- // Mapping strings to their lengths: 5, 3
- `Stream.of("apple", "pie").mapToInt(String::length).forEach(System.out::println);`
- // Mapping integers to their squares: 1, 4, 9
- `Stream.of(1, 2, 3).mapToInt(n -> n * n).forEach(System.out::println);`

mapToLong(ToLongFunction<? super T> mapper)

- Maps each element to a long, resulting in a LongStream
- // Mapping integers to their long values: 1, 2, 3
- Stream.of(1, 2, 3)
- .mapToLong(Integer::longValue).forEach(System.out::println);
- // Converting lengths of strings to a LongStream: 5, 6
- Stream.of("apple", "banana")
- .mapToLong(String::length).forEach(System.out::println);

mapToDouble(ToDoubleFunction<? super T> mapper)

- Maps each element to a double, resulting in a DoubleStream
- `// Mapping integers to their double values: 1.0, 2.0, 3.0`
- `Stream.of(1, 2, 3)`
- `.mapToDouble(Integer::doubleValue)`
- `.forEach(System.out::println);`
- `// Converting prices to a DoubleStream: 1.99, 2.99`
- `Stream.of("1.99", "2.99")`
- `.mapToDouble(Double::parseDouble)`
- `.forEach(System.out::println);`

Functional interfaces used with stream()

- // Terminal operations (often used to consume the stream):
 - Consumer void accept(T t); // Used with forEach
 - BinaryOperator T apply(T t1, T t2); // Used with reduce
 - Supplier T get(); // Used with collect
 - BiConsumer void accept(T t, U u); // Used with collect (accumulator in collectors)
 - Comparator int compare(T o1, T o2); // Used with sorted
 - Predicate boolean test(T t); // Used with filter
- // Intermediate operations (transform or map elements in the stream):
 - Function R apply(T t); // Used with map
 - ToIntFunction int applyAsInt(T value); // Used with mapToInt
 - ToLongFunction long applyAsLong(T value); // Used with mapToLong
 - ToDoubleFunction double applyAsDouble(T value); // Used with mapToDouble
 - UnaryOperator T apply(T t); // Used with map (when mapping to the same type)

The best way to learn

- Let's practice
 - Create a Province class
 - Create a Canada class with a list of Provinces
 - stream, filter, map, etc...
-
- Note: the next slides have stream methods related to parallel processing; we can ignore them for now, but they are included for the purpose of being complete

findAny()

- Returns any element in the stream, which can be useful with parallel streams for performance (in a non-deterministic way; i.e. random)
- `// Finding any even number: 2 (or 4)`
- `Optional<Integer> anyEven = Stream.of(1, 2, 3, 4)`
- `.parallel()`
- `.filter(n -> n % 2 == 0)`
- `.findAny();`
- `// Finding any name that starts with "C": Charlie`
- `Optional<String> anyC = Stream.of("Alice", "Bob", "Charlie")`
- `.parallel()`
- `.filter(name -> name.startsWith("C"))`
- `.findAny();`

splitter()

- Returns a Splitter, which can split the stream into multiple parts (generally, halves) for parallel processing
- // Using a splitter to traverse a stream
- `Splitter<String> splitter = Stream.of("Alice", "Bob").splitter();`
- `splitter.forEachRemaining(System.out::println);`
- // Splitting and processing a stream in parallel
- `Splitter<String> splitter = Stream.of("Alice", "Bob", "Charlie", "David")`
- `.splitter();`
- `Splitter<String> secondHalf = splitter.trySplit();`
- `secondHalf.forEachRemaining(System.out::println);`

parallel

- Converts the stream into a parallel stream, allowing for concurrent processing
- `// Making a stream parallel`
- `Stream.of(1, 2, 3, 4).parallel().forEach(System.out::println);`
- `// Using parallel for heavy processing`
- `Stream.of(1, 2, 3, 4).parallel().map(n -> n * n).forEach(System.out::println);`

sequential()

- Converts a parallel stream back into a sequential (non-parallel) stream
- `// Converting parallel back to sequential`
- `Stream.of(1, 2, 3, 4)`
- `.parallel()`
- `.sequential()`
- `.forEach(System.out::println);`
- `// Ensuring sequential order`
- `Stream.of("apple", "banana", "cherry")`
- `.parallel()`
- `.sequential()`
- `.forEach(System.out::println);`

unordered()

- Removes the order guarantee, useful for improving performance in parallel streams.
- `// Making a stream unordered`
- `Stream.of(1, 2, 3, 4).unordered().forEach(System.out::println);`
- Example 2: Using unordered in parallel processing
- `Stream.of("apple", "banana", "cherry")`
- `.parallel()`
- `.unordered()`
- `.forEach(System.out::println);`